

Documento de Evaluación de Viabilidad Técnica

CU-01: Asignar rol de usuario (RF-01)

- **Complejidad:** Media
- **Dependencias:** Ninguna (Proceso de registro inicial).
- **Observaciones técnicas:** Requiere implementar un esquema de base de datos que soporte RBAC (Role-Based Access Control). Se debe asegurar que la validación del rol se haga en el backend para evitar escalada de privilegios mediante manipulación de peticiones HTTP.

+1

CU-02: Cargar archivos de vacante (RF-02)

- **Complejidad:** Media
- **Dependencias:** CU-09 (Usuario debe estar autenticado).
- **Observaciones técnicas:** Implica manejo de *multipart/form-data*. Se debe definir una estrategia de nombramiento único (UUID) para evitar colisiones de archivos y configurar correctamente los permisos de escritura en el servidor o Bucket (S3/Blob).

+1

CU-03: Visualizar documentos del expediente (RF-03)

- **Complejidad:** Alta
- **Dependencias:** CU-02 (Deben existir archivos cargados).
- **Observaciones técnicas:** El requisito exige visualización "sin descarga". Esto requiere implementar librerías de renderizado en el navegador (como PDF.js para PDFs) o asegurar que el servidor envíe los encabezados **Content-Type** y **Content-Disposition: inline** correctos. Seguridad crítica para evitar XSS en archivos SVG/HTML.

+1

CU-04: Registrar comentarios de validación (RF-04)

- **Complejidad:** Baja
- **Dependencias:** CU-03 (Se comenta sobre un documento visualizado).
- **Observaciones técnicas:** Operación CRUD estándar. Se requiere una tabla relacional **Comentarios** vinculada al ID del documento y al ID del administrador. Importante sanitizar la entrada de texto para evitar inyecciones SQL o scripts.

+1

CU-05: Enviar postulación completa (RF-05)

- **Complejidad:** Media
- **Dependencias:** CU-02 (Carga de documentos).

- **Observaciones técnicas:** Funciona como un "Gatekeeper". Requiere una consulta agregada (`COUNT`) que compare los documentos subidos contra el catálogo de requisitos obligatorios antes de permitir el `UPDATE` del estado de la postulación.

CU-06: Consultar alertas de expediente (RF-06)

- **Complejidad:** Baja
- **Dependencias:** CU-04 o CU-10 (Cambios de estado previos).
- **Observaciones técnicas:** No requiere un sistema de notificaciones push complejo si se maneja como una vista filtrada en el dashboard. Se puede resolver con una consulta SQL indexada por estado (`WHERE status IN ('Rechazado', 'Pendiente')`).

CU-07: Filtrar listado de candidatos (RF-07)

- **Complejidad:** Baja
 - **Dependencias:** Ninguna (Visualización de datos existentes).
 - **Observaciones técnicas:** Implementación de filtrado dinámico en el backend. Se recomienda usar índices en la columna `vacante_id` de la base de datos para optimizar el tiempo de respuesta si el volumen de candidatos es alto.
- +1

CU-08: Validar características técnicas del archivo (RF-08)

- **Complejidad:** Media
 - **Dependencias:** Parte integral del CU-02.
 - **Observaciones técnicas:** La validación debe ser estricta en el backend (Server-side validation). No confiar solo en la extensión del nombre del archivo; se debe inspeccionar el "Magic Number" o encabezado del archivo binario para confirmar el tipo MIME real. Configurar límites de tamaño en el servidor web (Nginx/Apache) para evitar ataques DoS.
- +1

CU-09: Iniciar sesión con vigencia (RF-09)

- **Complejidad:** Media
 - **Dependencias:** Configuración previa de fechas de vacante.
 - **Observaciones técnicas:** Requiere un Middleware de autenticación personalizado. Además de verificar usuario/contraseña, debe consultar la fecha actual del servidor vs. la fecha de cierre de la convocatoria. Si `fecha_actual > fecha_cierre`, se bloquea el token JWT o la sesión.
- +1

CU-10: Desestimar candidatura (RF-10)

- **Complejidad:** Baja
- **Dependencias:** CU-05 (La postulación debe existir).

- **Observaciones técnicas:** Transición de máquina de estados. Simple actualización de campo en BD. Se sugiere registrar en un log de auditoría quién realizó la desestimación y la marca de tiempo (Timestamp) para trazabilidad.

CU-11: Visualizar actualización de estado (RF-11)

- **Complejidad:** Baja (Frontend) / Media (Sincronización).
- **Dependencias:** Cualquier CU que modifique estado (CU-04, CU-05, CU-10).
- **Observaciones técnicas:** Para cumplir con "inmediatamente", el frontend debe reactualizar el estado local tras recibir el código 200 OK del backend, sin necesidad de recargar toda la página (AJAX/Fetch). Asegurar consistencia eventual.

CU-12: Registrar dictamen final de contratación (RF-12)

- **Complejidad:** Media
- **Dependencias:** Todo el flujo de validación anterior completado.
- **Observaciones técnicas:** Acción crítica irreversible. Se recomienda implementar un patrón de "Doble confirmación" en el UI y una transacción atómica en base de datos para asegurar que no se generen estados inconsistentes. Solo accesible por el rol Administrador de mayor jerarquía.